1 Lecture 10 (Sep 29) - Recursion: Tower of Hanoi and Mergesort

> **Key idea** **Shrink, trust, combine:** Turn a big problem into smaller copies of the same problem. Trust that the copies get solved. Combine the answers.

The course now moves from *models of computation* (DFAs, grammars, Turing machines) to *algorithms*: how to solve problems **correctly** and **fast**.

1.1 The RealRAM Model

Same cost model as CS 225: anything a C-like language does cheaply is $O(1)$.

Operation	Cost
Arithmetic on a number	$O(1)$
Read or write `A[i]`	$O(1)$
Follow a pointer / reference	$O(1)$
Call a function (overhead)	$O(1)$

1.2 Describing Algorithms

What a "describe an algorithm" answer needs on homework and exams:

Give ✓	Don't give ×
Clear English or pseudocode	Real code (C++, Rust, Perl, …)
What each function **returns**	A correctness proof (if you follow a class pattern)
Running time: recurrence + solution	A loose bound "just to be safe"

> **Watch out** **Greedy algorithms need a formal proof:** They often look right and are wrong. Without a proof, graders assume a greedy algorithm is incorrect.

How answers are graded (target runtimes are not given):

Correct?	Speed	Credit
Yes	Faster than expected	Extra credit
Yes	Slower than expected	Big partial credit
No	Fast	Very little

> **Key idea** **Correctness first, then speed:**

1.3 Reductions and Recursion

A **reduction** solves problem A by using a solver for problem B as a **black box** (you never look inside it):

instance of A → black box solves B → answer for A

Recursion is a reduction where B is **a smaller copy of A itself**:

instance of A size n → smaller instance(s) of A → recursion fairy solves them → combine into answer

If the instance is too small to shrink, answer it directly. That is the **base case**.

1.3.1 Example: sum of an array

```
Sum(A[1..n]):
  if n = 0
    return 0                          ⟨⟨base case⟩⟩
  return Sum(A[1..n - 1]) + A[n]      ⟨⟨trust the smaller call⟩⟩
```

Sum(3 | 1 | 4) = Sum(3 | 1) +4 = 4 + 4 = 8

Blue part: handed to the recursion fairy, who returns 4. Yellow part: our own work.

1.3.2 The recursion fairy

> **Key idea** Never trace what happens **inside** a recursive call. Assume it returns the right answer. Tracing the calls of the calls overflows your brain.

Why trusting is allowed: **induction** (Lecture 01). Assume the algorithm is correct on every smaller input (inductive hypothesis). Then show it is correct on size n.

Your three jobs:

1. Every recursive call is on a **strictly smaller** instance of the **same** problem
2. Every **base case** is answered correctly
3. The **combine step** is correct, assuming the recursive calls were correct

1.4 Tower of Hanoi

Puzzle by Édouard Lucas (1883). Move the whole stack from `src` to `dst`.

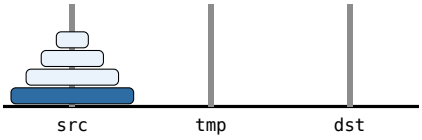


Rules:

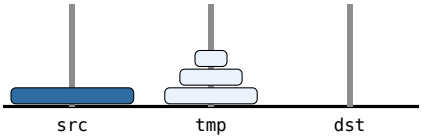
- Move **one** disk at a time, from the top of a pile to the top of another peg
- **Never** put a bigger disk on a smaller one

1.4.1 The recursive idea

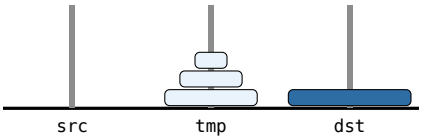
The biggest disk (dark) must move to `dst` at some point. To move it, every smaller disk must be out of the way, stacked on `tmp`.



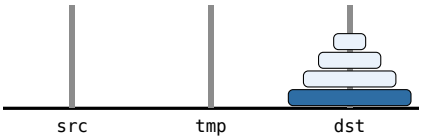
Start



Step 1: recursively move $n-1$ disks to `tmp`



Step 2: move the biggest disk to `dst`



Step 3: recursively move $n-1$ disks onto it

> **Key idea** The big disk never gets in the way: any disk may sit on it. So moving the $n-1$ small disks is **the same puzzle**, just smaller, with a different target peg.

1.4.2 Algorithm

State what the call does, so a reader can check it:

```
Hanoi(n, src, dst, tmp):   ⟨⟨move disks 1..n from src to dst⟩⟩
  if n > 0
    Hanoi(n - 1, src, tmp, dst)     ⟨⟨step 1⟩⟩
    move disk n from src to dst     ⟨⟨step 2⟩⟩
    Hanoi(n - 1, tmp, dst, src)     ⟨⟨step 3⟩⟩
```

Base case: $n = 0$, nothing to move.

1.4.3 Counting moves

Let $T(n)$ be the number of moves for n disks. Two recursive calls, plus one move:

$$T(n) = \begin{cases} 0 & \text{if } n = 0 \\ 2T(n-1) + 1 & \text{if } n > 0 \end{cases}$$

This is a **recurrence**: the cost is written in terms of the cost of smaller inputs. Compute small values and spot the pattern:

n	0	1	2	3	4	5
$T(n)$	0	1	3	7	15	31
$2^n - 1$	0	1	3	7	15	31

$$\boxed{T(n) = 2^n - 1}$$

1.4.4 Proof by induction

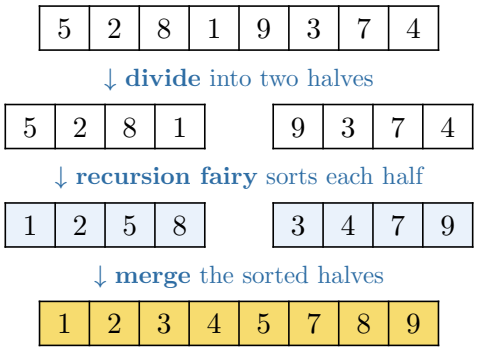
Guess-and-verify: check the guess with induction. Assume $T(k) = 2^k - 1$ for all $k < n$.

- $n = 0$: $T(0) = 0 = 2^0 - 1$ ✓
- $n > 0$:

$$\begin{aligned} T(n) &= 2T(n-1) + 1 && \text{definition of } T \\ &= 2(2^{n-1} - 1) + 1 && \text{inductive hypothesis} \\ &= 2^n - 1 && \text{arithmetic} \end{aligned}$$

1.5 Mergesort

Sort $A[1..n]$ into increasing order. By von Neumann (1945).




```
MergeSort(A[1..n]):
  if n > 1
    m ← ⌊n/2⌋
    MergeSort(A[1..m])
    MergeSort(A[m+1..n])
    Merge(A[1..n], m)
```

Base case: $n \leq 1$. Already sorted, nothing to do.

1.5.1 Merging

Both halves are sorted, so the smallest remaining element is always at the **front** of one half. Repeatedly compare the two fronts and take the smaller one:

Left half	Right half	Take	Output so far
1 2 5 8	3 4 7 9	1	1
2 5 8	3 4 7 9	2	1 2
5 8	3 4 7 9	3	1 2 3
5 8	4 7 9	4	1 2 3 4
5 8	7 9	5	1 2 3 4 5
⋮	⋮	⋮	⋮

Each comparison places one element, so `Merge` takes $\boldsymbol{O(n)}$ time. One way to write it:

```
Merge(A[1..n], m):
  i ← 1;  j ← m + 1          ⟨⟨fronts of the two halves⟩⟩
  for k ← 1 to n
    if j > n                 ⟨⟨right half used up⟩⟩
      B[k] ← A[i];  i ← i + 1
    else if i > m            ⟨⟨left half used up⟩⟩
      B[k] ← A[j];  j ← j + 1
    else if A[i] < A[j]
      B[k] ← A[i];  i ← i + 1
    else
      B[k] ← A[j];  j ← j + 1
  for k ← 1 to n
    A[k] ← B[k]
```

1.5.2 Recurrence

Two half-size recursive calls, plus $O(n)$ for the merge:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

> **Key idea** Floors and ceilings ($\lfloor \frac{n}{2} \rfloor$ vs $\lceil \frac{n}{2} \rceil$) don't matter when the problem shrinks by a constant fraction. Just write $\frac{n}{2}$.

1.6 Recursion Trees

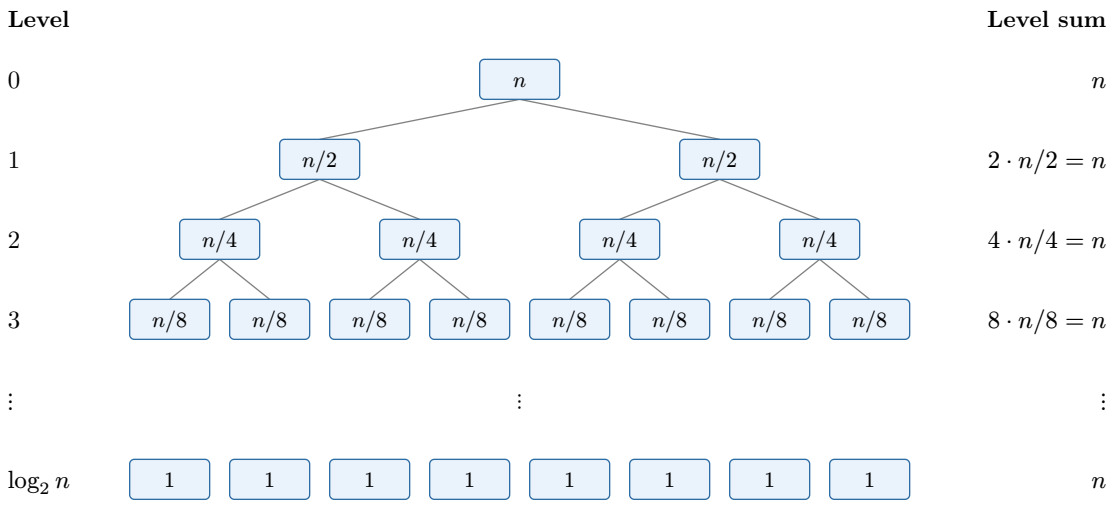
A picture of every recursive call, used to **solve** recurrences.

- One **node** per recursive call. The **root** is the first call.
- A node's **children** are the recursive calls it makes.
- A node's **value** is the work done in that call, **not counting** its children.

Key idea **Total time = sum of all node values:** Add up each level (row) first, then add up the levels.

1.6.1 Mergesort

The root does n work (the merge). Each child has half the elements, so it does $\frac{n}{2}$ work. Each grandchild does $\frac{n}{4}$. And so on, down to size 1.



- **Every level sums to n:**
- **How many levels?** Each level halves the size: $n \to \frac{n}{2} \to \frac{n}{4} \to \ldots \to 1$. You can halve n only $\log_2 n$ times before reaching 1.

$$T(n) = \underbrace{n}_{\text{per level}} \times \underbrace{\log_2 n}_{\text{levels}} = \boxed{O(n \log n)}$$

The log base doesn't matter inside big-O (changing base is a constant factor).

1.6.2 The three common shapes

Look at how the level sums change from top to bottom:



total = level sum × levels
e.g. mergesort: $O(n \log n)$

root dominates
total = O(root)

leaves dominate
total = O(# leaves)

Shrinking and growing are **geometric series**: each level is the one above times a fixed ratio $\neq 1$ (for example $n, \frac{n}{3}, \frac{n}{9}, \ldots$). A geometric series is proportional to its **biggest term**, however many terms it has. Lecture 11 has examples of both.

Watch out Don't memorize a formula theorem instead. It only fits recurrences with equal-size pieces. Lecture 11's recurrences don't fit it, but recursion trees still work.